



2D Potential Fields on a Mobile Robot: Attraction, Repulsion and Traps (Hands on)

Ankit Ravankar
Tohoku University, Japan
Hands-on Session 2
2nd October 2026

From 1D to 2D

Session 1 controlled one number. Today the robot controls a position in the plane.

Session 1 · 1D

State	distance d to the box
Sensor	LiDAR front distance
Forces	scalars along one line
Output	linear speed v only
Lesson	equilibrium, damping regimes



Session 2 · 2D (today)

State	position $p = (x, y)$
Sensor	odometry pose (obstacles known)
Forces	vectors $F = (F_x, F_y)$
Output	speed v and turn rate ω
Lesson	local minima, saddles, escape

Same idea, one more dimension: the force becomes a vector, so the robot must also steer to follow it.

Problem

How does a robot reach a goal in a plane full of obstacles?



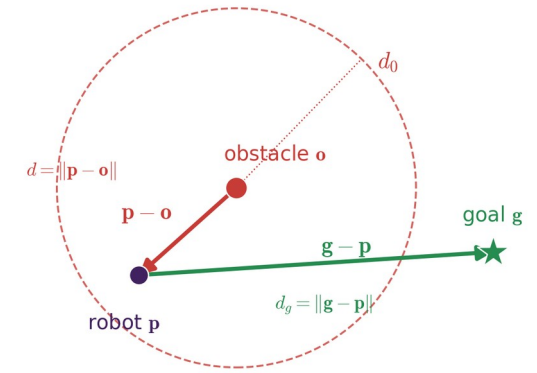
Move toward the goal



Go around obstacles, not just stop



Choose direction and speed together

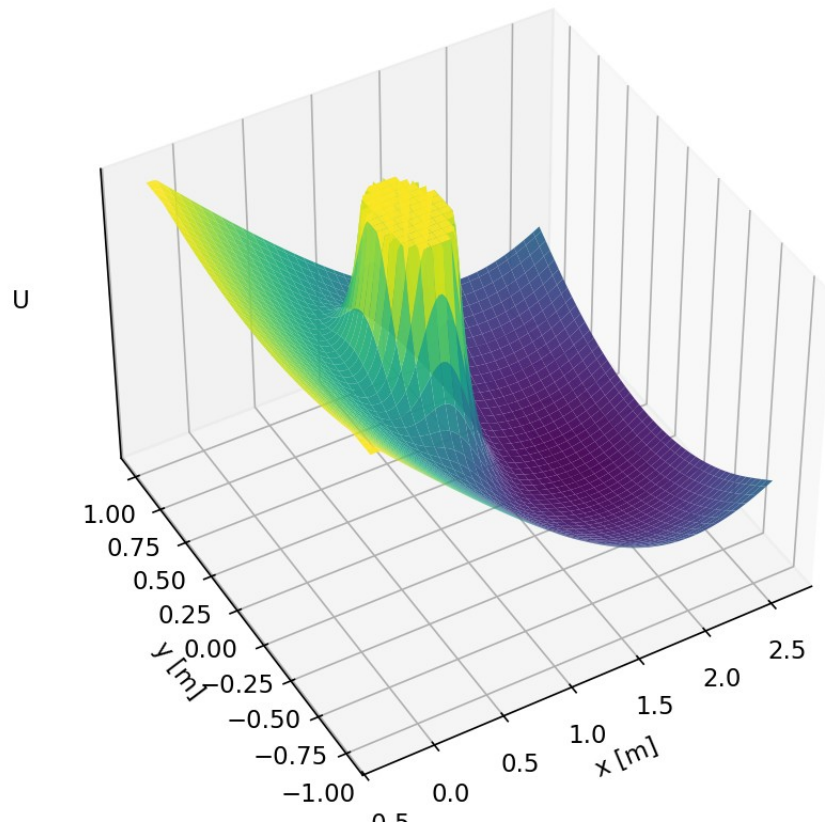


Potential Fields — one framework for all three: compute a force everywhere, then follow it.

Concept

Think of the robot as a ball on a hilly 2D landscape. It always moves to reduce the potential energy $U(x, y)$.

Potential energy landscape — the robot rolls downhill to the goal



Hill (obstacle)

High potential — pushes the robot away

Valley (goal)

Lowest potential — pulls the robot in

Ball

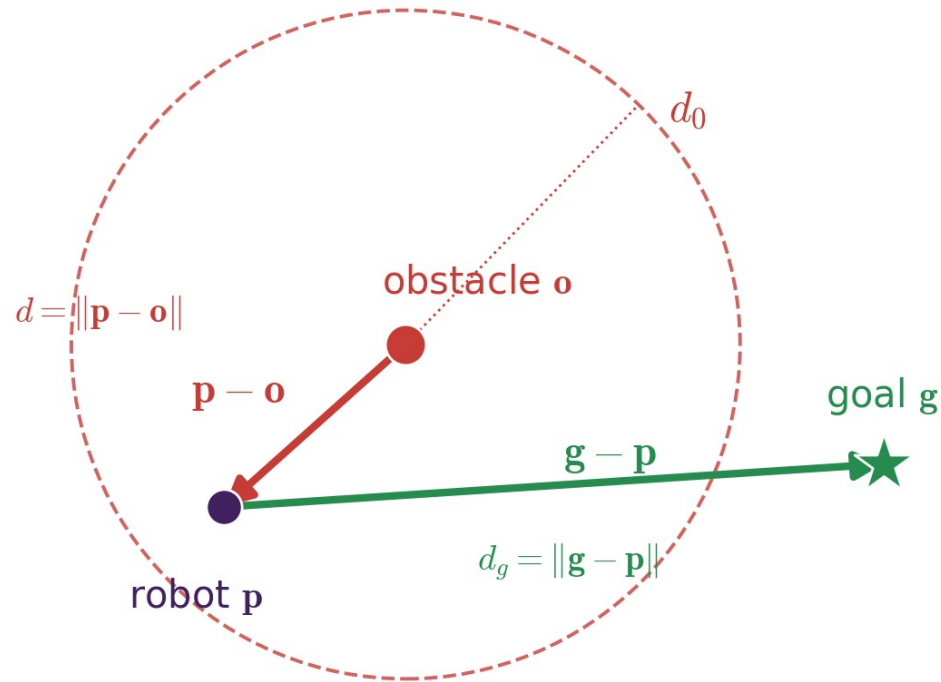
The robot, at position $p = (x, y)$

Rolling downhill

Always moves to reduce $U(x, y)$

Notation

Everything is measured in metres in the odom frame, starting from where the robot boots: (0, 0).



$\mathbf{p} = (x, y)$ robot position (from /odom)

$\mathbf{g}$ goal position (from the YAML file)

$\mathbf{o}_i$ obstacle i (from the YAML file)

d_g distance robot $\rightarrow$ goal

d_i distance robot $\rightarrow$ obstacle i

d_0 influence radius: repulsion only if $d < d_0$

$$d_g = \|\mathbf{g} - \mathbf{p}\| \quad d_i = \|\mathbf{p} - \mathbf{o}_i\|$$

Potential energy $U(x, y)$

$$U(\mathbf{p}) = U_{\text{att}}(\mathbf{p}) + \sum_i U_{\text{rep},i}(\mathbf{p})$$

Attractive potential U_{att}

$$U_{\text{att}}(\mathbf{p}) = \begin{cases} \frac{1}{2} k_{\text{att}} d_g^2 & d_g \leq 1 \\ k_{\text{att}} d_g - \frac{1}{2} k_{\text{att}} & d_g > 1 \end{cases}$$

Bowl near the goal, cone far away.
Minimum exactly at the goal.

Repulsive potential U_{rep}

$$U_{\text{rep}}(\mathbf{p}) = \begin{cases} \frac{1}{2} k_{\text{rep}} \left(\frac{1}{d} - \frac{1}{d_0} \right)^2 & d < d_0 \\ 0 & d \geq d_0 \end{cases}$$

Steep hill around each obstacle.
Exactly zero beyond d_0 ; grows to ∞ at the obstacle.

The total landscape is simply the sum: one bowl, plus one hill per obstacle.

Force = slope of the landscape

The force points downhill: the direction in which U decreases fastest.

$$\mathbf{F}(\mathbf{p}) = -\nabla U(\mathbf{p})$$

ATTRACTIVE

$$\nabla d_g = -\frac{\mathbf{g} - \mathbf{p}}{d_g}$$

near the goal: $\nabla\left(\frac{1}{2}k d_g^2\right) = k d_g \nabla d_g = -k(\mathbf{g} - \mathbf{p}) \Rightarrow \mathbf{F}_{\text{att}} = k_{\text{att}} (\mathbf{g} - \mathbf{p})$

far from goal: $\nabla(k d_g) = k \nabla d_g \Rightarrow \mathbf{F}_{\text{att}} = k_{\text{att}} \frac{\mathbf{g} - \mathbf{p}}{\|\mathbf{g} - \mathbf{p}\|}$

REPULSIVE

$$\frac{dU_{\text{rep}}}{dd} = -k_{\text{rep}} \left(\frac{1}{d} - \frac{1}{d_0} \right) \frac{1}{d^2} \quad \nabla d = \frac{\mathbf{p} - \mathbf{o}}{d}$$

chain rule: $\mathbf{F}_{\text{rep}} = -\frac{dU_{\text{rep}}}{dd} \nabla d = k_{\text{rep}} \left(\frac{1}{d} - \frac{1}{d_0} \right) \frac{1}{d^2} \frac{\mathbf{p} - \mathbf{o}}{d}$

*two minus signs cancel $\rightarrow$
points away from the obstacle*

The force equations in the code

Attractive force

$$\mathbf{F}_{\text{att}} = \begin{cases} k_{\text{att}} (\mathbf{g} - \mathbf{p}) & d_g \leq 1 \\ k_{\text{att}} \frac{\mathbf{g} - \mathbf{p}}{\|\mathbf{g} - \mathbf{p}\|} & d_g > 1 \end{cases}$$

Repulsive force (per obstacle)

$$\mathbf{F}_{\text{rep}} = \underbrace{k_{\text{rep}} \left(\frac{1}{d} - \frac{1}{d_0} \right) \frac{1}{d^2}}_{\text{magnitude}} \underbrace{\frac{\mathbf{p} - \mathbf{o}}{d}}_{\text{direction}}, \quad d < d_0$$

$$\mathbf{F} = \mathbf{F}_{\text{att}} + \sum_i \mathbf{F}_{\text{rep},i}$$

`attractive_force(pos)`

$\mathbf{F}_{\text{att}}$ — pulls toward $\mathbf{g}$

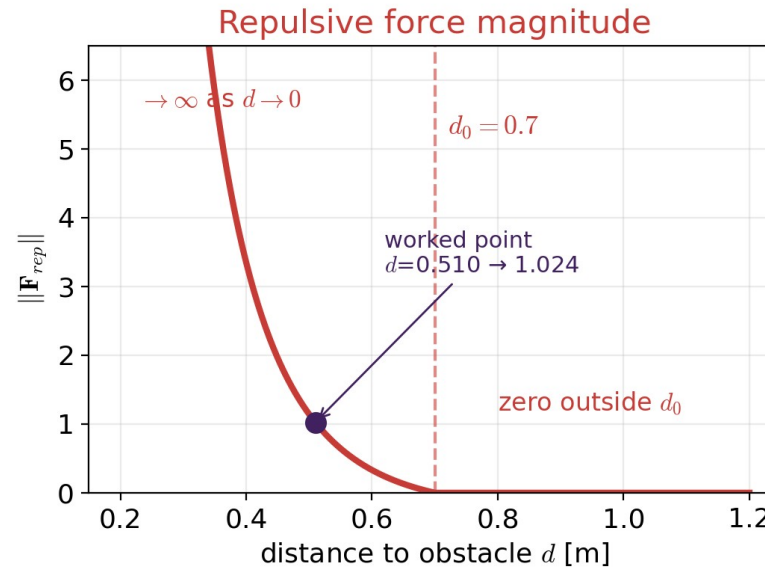
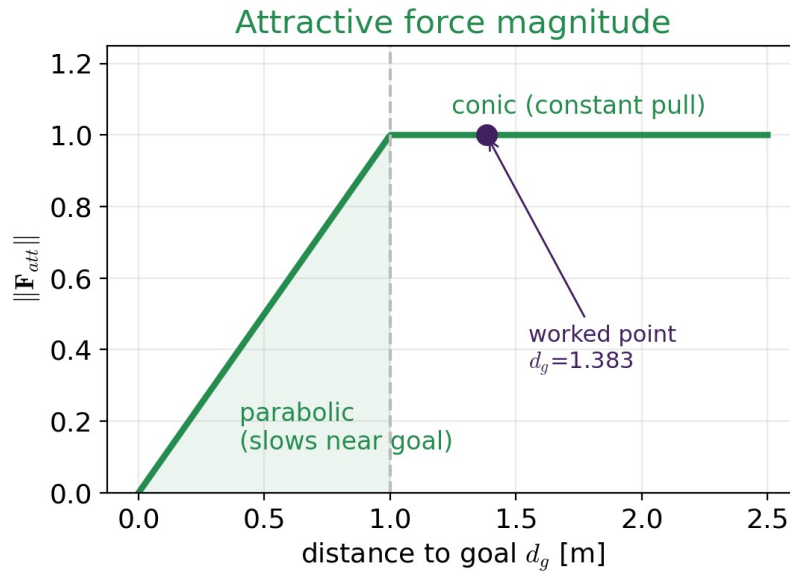
`repulsive_force(pos)`

$\sum \mathbf{F}_{\text{rep}}$ — one push per obstacle

`total_force(pos)`

$\mathbf{F}$ — what the robot follows

Why these shapes?



Constant pull far away

The robot does not race toward a distant goal: $|F_{att}| = k_{att}$.

Soft arrival near the goal

Inside 1 m the pull fades linearly to zero.

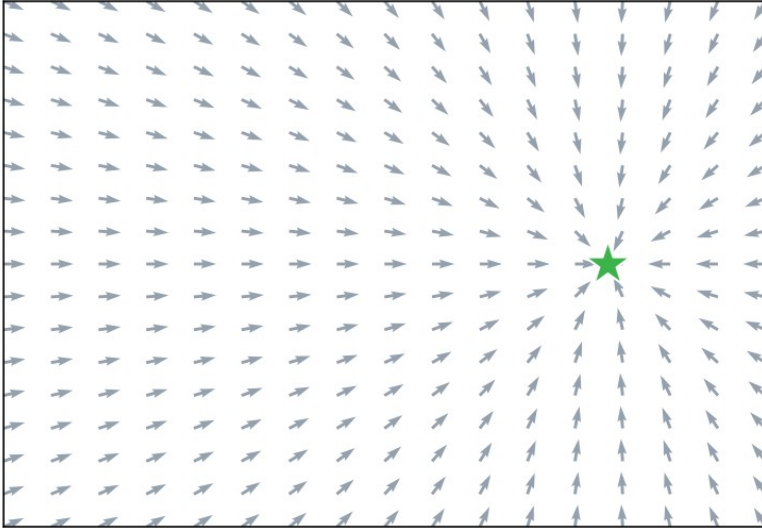
Repulsion is local

Zero outside d_0 , explodes as $d \rightarrow 0$.
Safety by design.

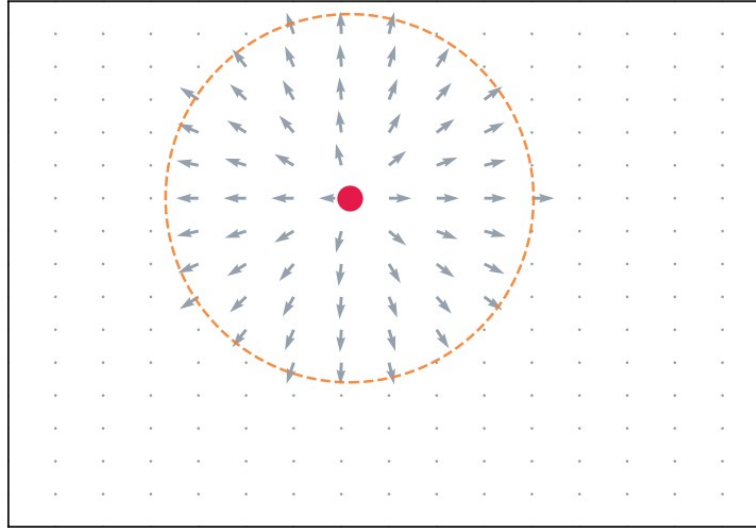
Worked point (next): $d_g = 1.383 \text{ m} \rightarrow |F_{att}| = 1.00$ $d = 0.510 \text{ m} \rightarrow |F_{rep}| = 1.024$

Vector sum of two fields

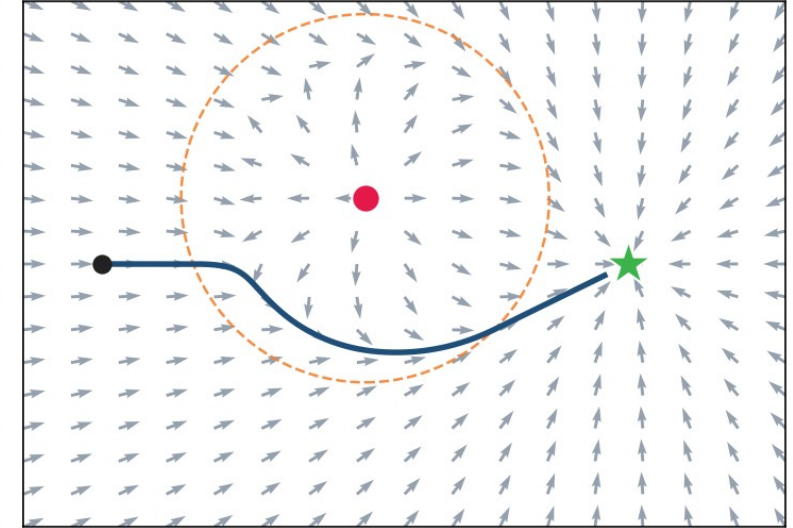
Attractive field
(goal pulls)



Repulsive field
(obstacle pushes)



Total = attractive + repulsive



Each grey arrow shows only the direction of F at that point. The blue path is what you get by always following the arrows.

Key insight: nobody planned the curve. It appears because the robot follows the sum of two simple fields.

Worked example · Step 1 — Where are we?

SCENARIO 1 · SINGLE OBSTACLE

$k_{att} = 1.0$ | $k_{rep} = 0.5$ | $d_o = 0.7$ m | $g = (2.0, 0.0)$ | $o = (1.0, 0.25)$ | $p = (0.62, -0.09)$

1a · Distance to the goal

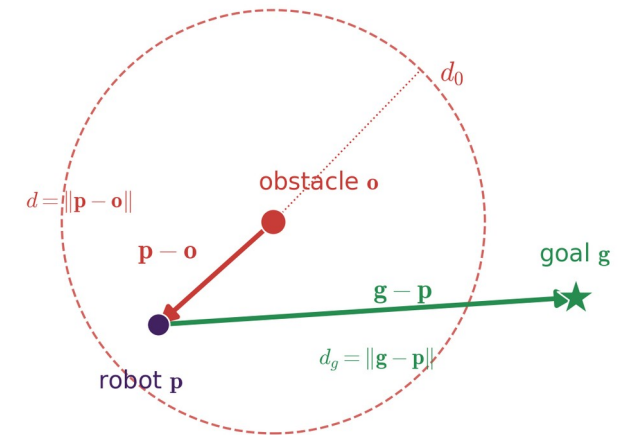
$$g - p = (2.00 - 0.62, 0.00 - (-0.09)) = (1.38, 0.09)$$

$$d_g = \sqrt{1.38^2 + 0.09^2} = 1.383 \text{ m} > 1 \Rightarrow \text{conic branch}$$

1b · Distance to the obstacle

$$p - o = (0.62 - 1.00, -0.09 - 0.25) = (-0.38, -0.34)$$

$$d = \sqrt{0.38^2 + 0.34^2} = 0.510 \text{ m} < d_o = 0.7 \Rightarrow \text{repulsion ON}$$



Two quick decisions before any force: which branch of F_{att} , and is the obstacle close enough to matter?

Worked example · Steps 2 & 3 — Each force

Step 2 · Attractive (conic branch, unit vector toward g)

$$\mathbf{F}_{\text{att}} = 1.0 \cdot \frac{(1.38, 0.09)}{1.383} = (0.998, 0.065) \quad \|\mathbf{F}_{\text{att}}\| = 1.00$$

Step 3 · Repulsive

(a) the three terms

$$\frac{1}{d} = 1.961 \quad \frac{1}{d_0} = 1.429 \quad \frac{1}{d^2} = 3.846$$

(b) the magnitude

$$m = 0.5 \times (1.961 - 1.429) \times 3.846 = 0.5 \times 0.533 \times 3.846 = 1.024$$

(c) the direction (unit vector away from o)

$$\hat{\mathbf{n}} = \frac{(-0.38, -0.34)}{0.510} = (-0.745, -0.667)$$

(d) the force

$$\mathbf{F}_{\text{rep}} = 1.024 \times (-0.745, -0.667) = (-0.763, -0.683)$$

Sanity check

$|\mathbf{F}_{\text{rep}}| = 1.024$ is larger than $|\mathbf{F}_{\text{att}}| = 1.00$. Here the obstacle wins, so the robot will be pushed away from it.

Direction check

$\hat{\mathbf{n}}$ points from o to p: down-left.
Repulsion always points away from the obstacle.

Worked example · Step 4 — Add them up

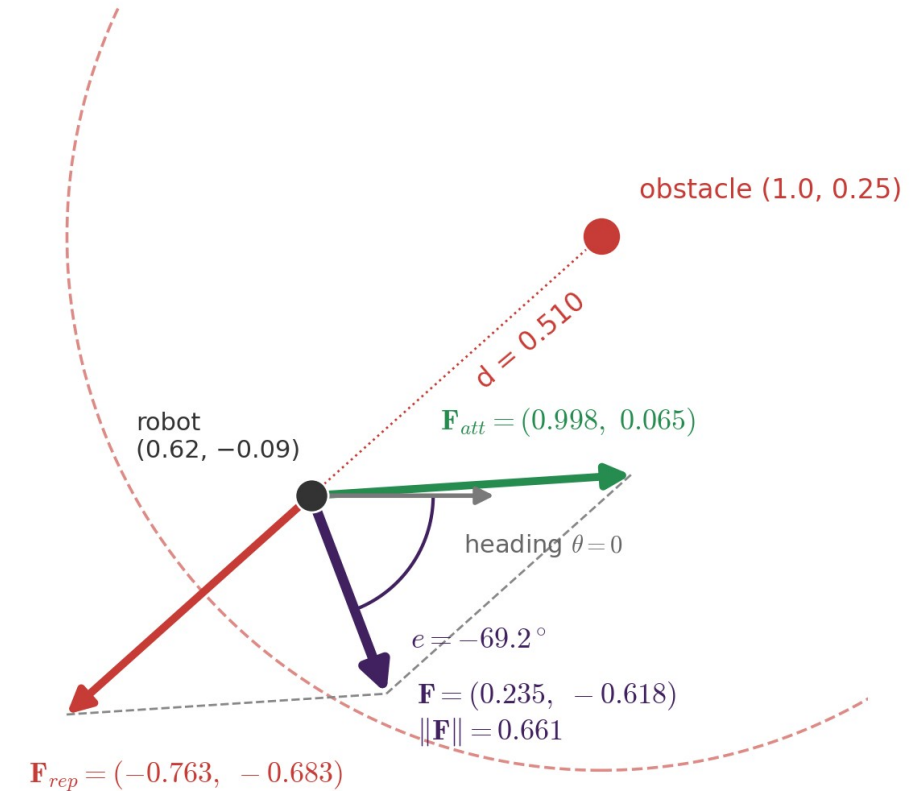
$$\mathbf{F} = (0.998 - 0.763, 0.065 - 0.683) = (0.235, -0.618)$$

$$\|\mathbf{F}\| = \sqrt{0.235^2 + 0.618^2} = 0.661$$

What it means

The repulsion cancels most of the forward pull and adds a strong sideways component. The robot is told to head 69° to the right of its current heading.

Check it live: the node prints $|\mathbf{F}|$ every cycle. Find this position in your log and compare.



From a force to the wheels

The TB3 is a differential drive: it cannot move sideways. It can only drive forward (v) and turn (ω).

UNICYCLE MODEL

$$\dot{x} = v \cos \theta \quad \dot{y} = v \sin \theta \quad \dot{\theta} = \omega$$



**Safety guards
in the code →**

Deadband

$|e| < 0.05 \rightarrow \omega = 0$
no twitching when aligned

STEERING LAW

1 $\theta_{\text{des}} = \text{atan2}(F_y, F_x)$

Which way does the field point?

2 $e = \text{wrap}(\theta_{\text{des}} - \theta) \in (-\pi, \pi]$

How far off is my heading?

3 $\omega = \text{sat}_{\omega_{\text{max}}}(K_h e)$

Turn toward it ($K_h = 1.5$, $\omega_{\text{max}} = 1.0$ rad/s)

4 $v = v_{\text{max}} \max(0, \cos e)$

Drive forward only when facing it

Turn latch

$|e| > 2.8 \rightarrow$ pick one side
no left-right dithering at 180°

Slew limit

$|\Delta\omega| \leq 0.25$ / cycle
no jerks

Worked example · Step 5 — Wheel commands

Robot heading $\theta = 0$ (facing +x) | $K_h = 1.5$ | $\omega_{max} = 1.0$ rad/s | $v_{max} = 0.15$ m/s

1 desired heading

$$\theta_{des} = \text{atan2}(-0.618, 0.235) = -1.208 \text{ rad } (-69.2^\circ)$$

2 heading error

$$e = -1.208 - 0 = -1.208 \text{ rad}$$

3 turn rate

$$\omega = \text{sat}_{1.0}(1.5 \times (-1.208)) = \text{sat}_{1.0}(-1.81) = -1.0 \text{ rad/s}$$

4 forward speed

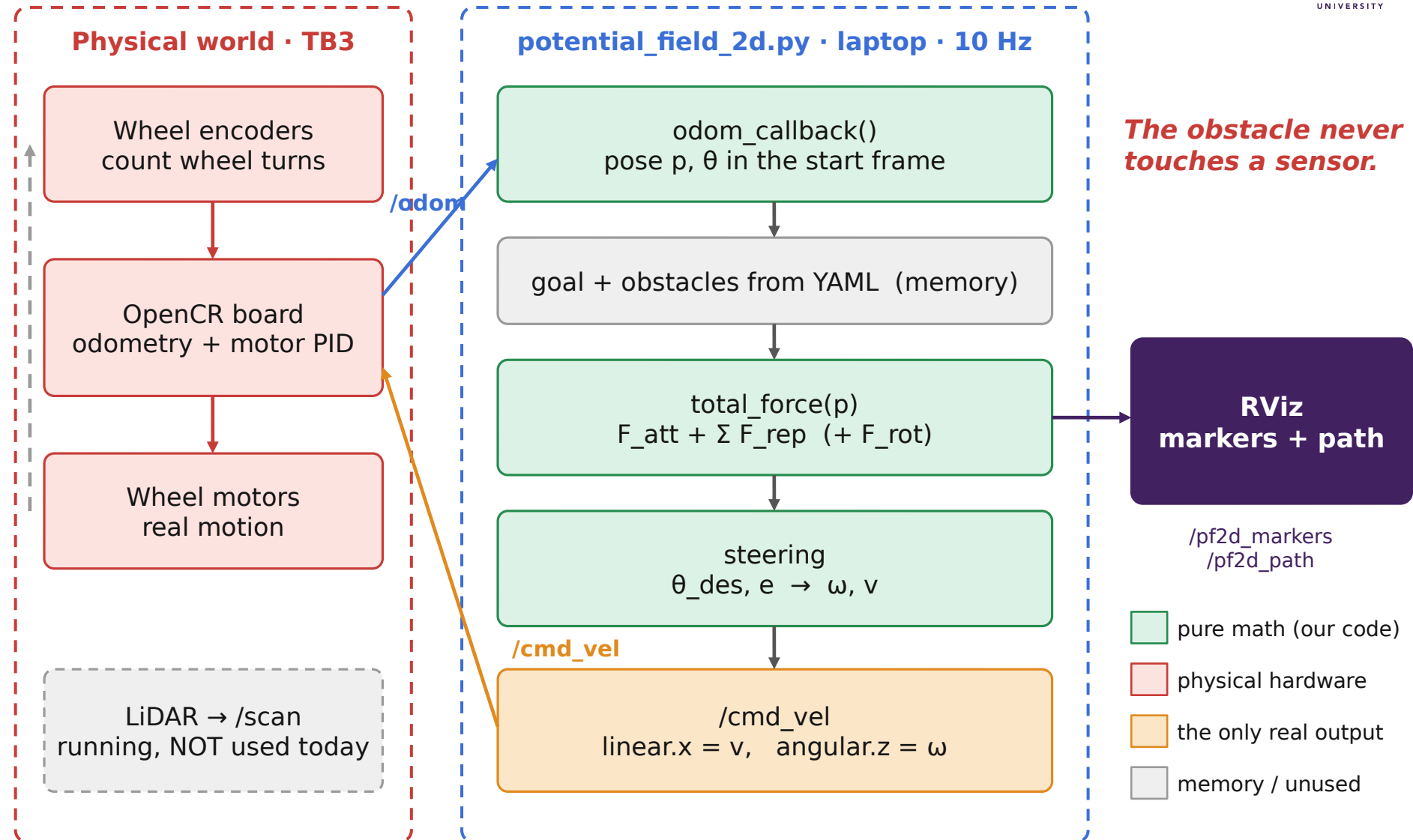
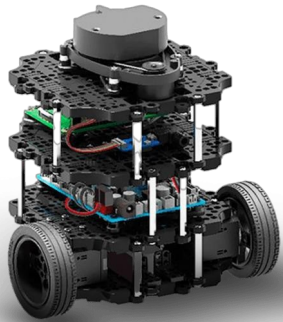
$$v = 0.15 \times \cos(1.208) = 0.15 \times 0.355 = 0.053 \text{ m/s}$$

5 over the next 0.1 s

$$\Delta\theta = \omega \Delta t = -0.10 \text{ rad } (-5.7^\circ) \quad \Delta x \approx v \Delta t = 5.3 \text{ mm}$$

Result: turn hard right at the ω limit while creeping forward at about a third of full speed — the curve you see the robot start in RViz.

What is happening on the TB3



Inside one control cycle (every 0.1 s)

- 1 Read pose** `odom_callback()`
 $p = (x, y), \theta$ relative to the start
- 2 Goal reached?** $d_g < 0.15 \text{ m}$
 stop, publish zero `/cmd_vel`
- 3 Compute force** `total_force(p)`
 $F_{att} + \sum F_{rep} (+ F_{rot})$
- 4 Stuck?** $|F| < 0.05$ for 15 cycles
 print LOCAL MINIMUM, stop
- 5 Heading** `atan2(F_y, F_x)`
 $e = \text{wrap}(\theta_{des} - \theta)$
- 6 Turn rate** $\omega = \text{sat}(1.5 \cdot e)$
 + deadband, latch, slew
- 7 Speed** $v = 0.15 \cdot \max(0, \cos e)$
 slow while turning
- 8 Publish** `/cmd_vel`
 + `/pf2d_markers`, `/pf2d_path`, log line

Reading the log line (format, values from our worked example):

`t= 5.9s | pos=( 0.62, -0.09) | |F|= 0.66 | dg=1.38 | v=0.05 w=-1.00`

time · where the robot is · force magnitude · distance to goal · the two wheel commands

What controls what?

PARAMETER GUIDE

Change one at a time — predict before you run!

Parameter	Too small →	Too large →	Baseline
k_att attractive gain	Weak pull, traps form easily	Cuts close, avoidance weakens	1.0
k_rep repulsive gain	Passes close to obstacles	Wide detours, refuses gaps	0.5
d_o influence radius	Reacts late, sharp swerves	Fields overlap → more minima	0.7 m
k_rot rotational gain	Too weak to leave a saddle	Circles obstacles, wanders	0.5
max_lin_speed v_max	Slow but precise	Wide turns, more odom drift	0.15
heading_gain K _h	Sluggish turns, wide arcs	Wobbles around the heading	1.5
goal_tolerance arrival radius	Jitters, never 'arrives'	Stops far from the goal	0.15 m

Equilibrium in 2D — where $F = 0$

$$\mathbf{F}_{\text{att}}(\mathbf{p}^*) + \sum_i \mathbf{F}_{\text{rep},i}(\mathbf{p}^*) = \mathbf{0}$$

WORKED EXAMPLE · OBSTACLE DEAD AHEAD (SCENARIO 4)

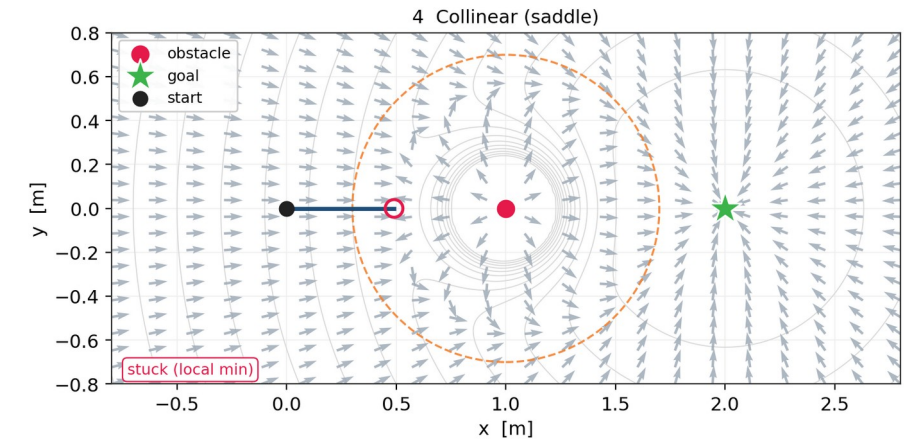
On the start-goal line both forces are horizontal, so the vector equation becomes one scalar equation:

$$k_{\text{att}} = k_{\text{rep}} \left(\frac{1}{d^*} - \frac{1}{d_0} \right) \frac{1}{d^{*2}}$$

$$1.0 = 0.5 \left(\frac{1}{d^*} - 1.429 \right) \frac{1}{d^{*2}} \implies d^* = 0.512 \text{ m}$$

$$x^* = 1.000 - 0.512 = 0.488 \text{ m}$$

$$\text{check: } 0.5 \times (1.953 - 1.429) \times 3.814 = 1.000 \checkmark$$

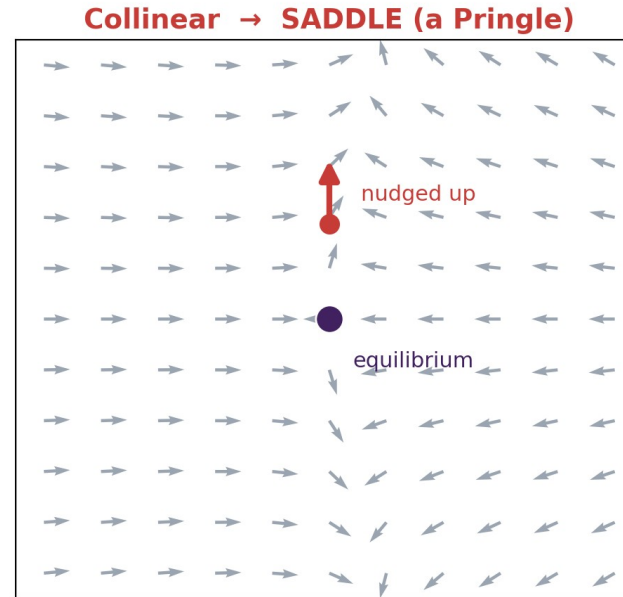
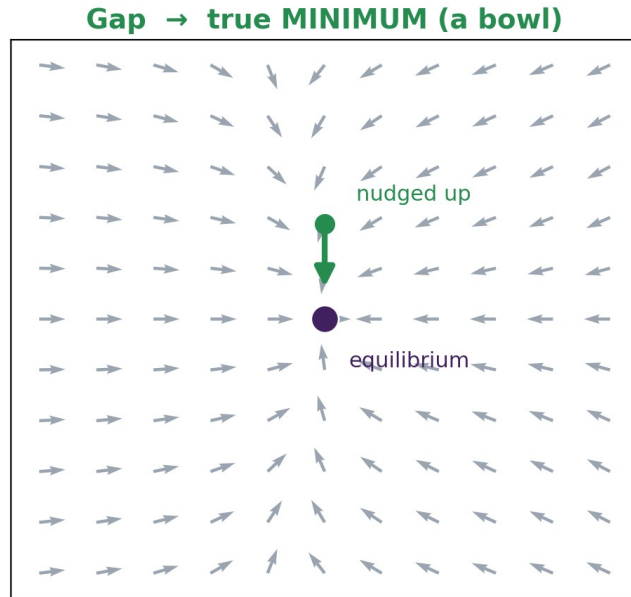


$$\text{Session 1: } k_{\text{att}}(d_{\text{eq}} - d_{\text{goal}}) = k_{\text{rep}} \left(\frac{1}{d_{\text{eq}}} - \frac{1}{d_0} \right) \frac{1}{d_{\text{eq}}^2}$$

Same structure as Session 1.

The robot stops at $x^* = 0.488 \text{ m}$ — 0.51 m short of the obstacle, with the goal still 1.5 m away.

Not all equilibria are equal



Minimum = a bowl

Every direction is uphill. Noise cannot get the robot out. (Gap, box canyon.)

Saddle = a Pringle

Uphill along the axis, downhill sideways. Any noise tips it off. (Obstacle dead ahead.)

On the real robot

Scenario 4 often escapes on its own: odometry noise supplies the nudge.

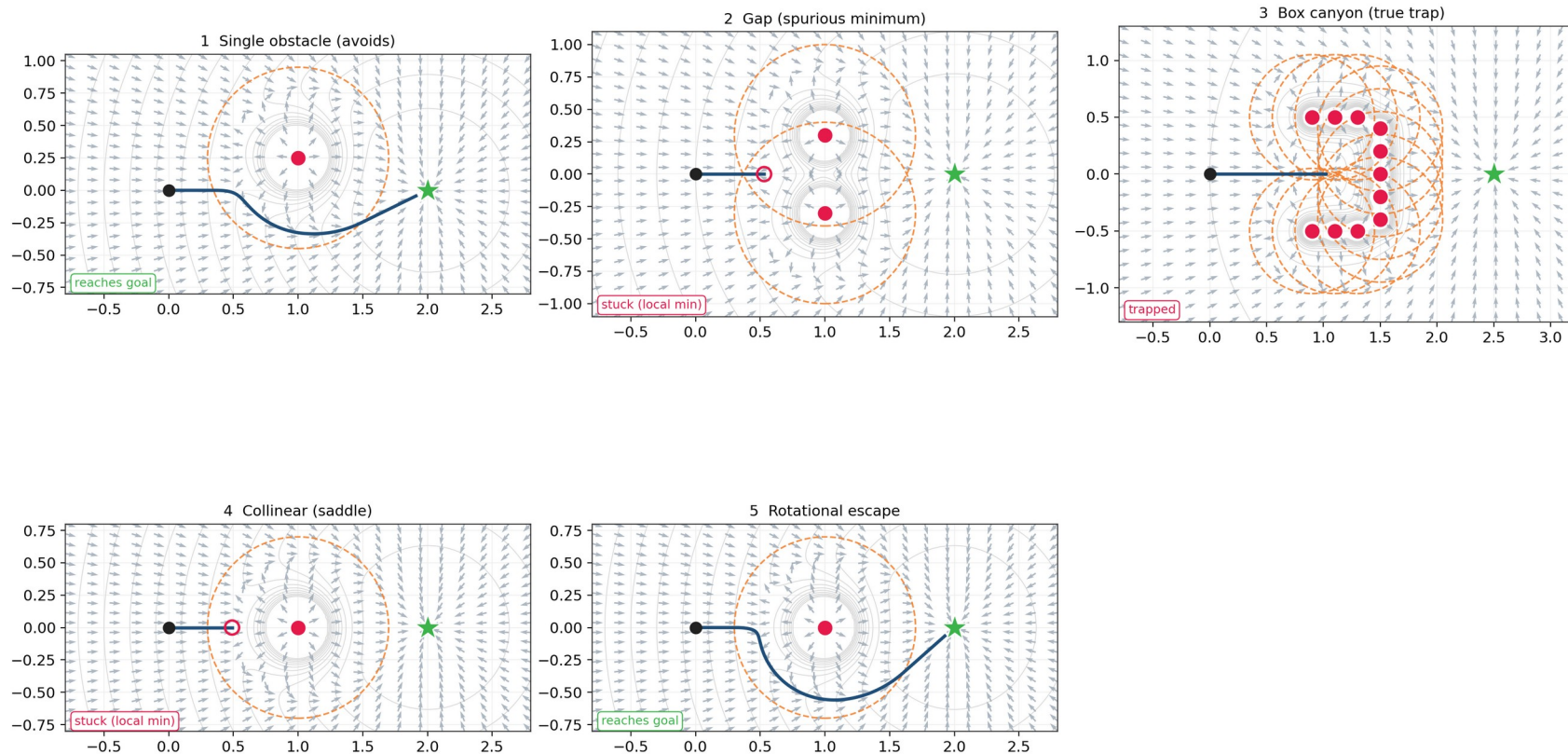
Gap: $\mathbf{F}(0.531, +0.05) = (-0.037, -0.167) \Rightarrow$ pushed back

Collinear: $\mathbf{F}(0.488, +0.05) = (0.031, +0.062) \Rightarrow$ pushed away

The test: nudge the robot 5 cm sideways and see which way the force points.

Five scenarios, one condition

Potential Fields in 2D — the five scenarios



- 1 Single obstacle**
reaches the goal
- 2 Gap between two**
stuck at $x \approx 0.53$ (minimum)
- 3 Box canyon**
trapped at $x \approx 1.02$ (minimum)
- 4 Obstacle dead ahead**
balances at $x \approx 0.49$ (saddle)
- 5 Same + rotational field**
escapes to the goal

Escaping a saddle: the rotational field

$$\mathbf{F}_{\text{rot}} = k_{\text{rot}} \left(\frac{1}{d} - \frac{1}{d_0} \right) \hat{\mathbf{t}}, \quad \hat{\mathbf{t}} = (-\hat{n}_y, \hat{n}_x)$$

WORKED EXAMPLE · $\mathbf{p} = (0.50, 0)$

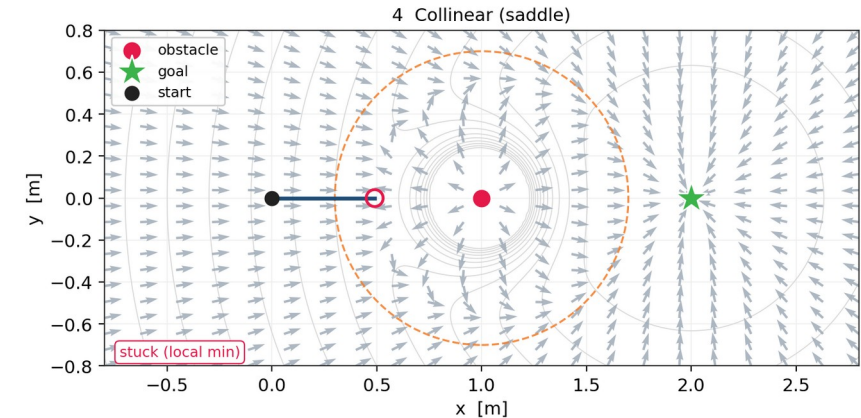
$$\mathbf{p} = (0.50, 0) : \quad d = 0.50, \quad \hat{\mathbf{n}} = (-1, 0), \quad \hat{\mathbf{t}} = (0, -1)$$

$$\mathbf{F}_{\text{rot}} = 0.5 (2.000 - 1.429) (0, -1) = (0, -0.286)$$

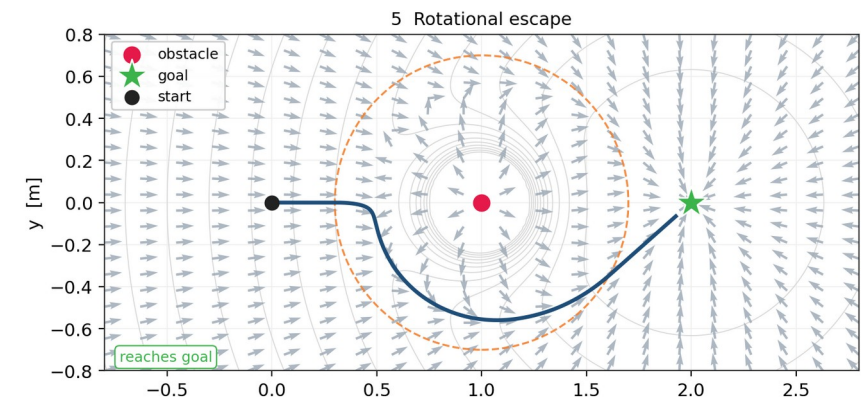
$$\mathbf{F} = (1, 0) + (-1.143, 0) + (0, -0.286) = (-0.143, -0.286) \Rightarrow \theta_{\text{des}} = -116.6^\circ$$

Without $\mathbf{F}_{\text{rot}}$ the force would be $(-0.143, 0)$: straight back, no reason to turn. The tangential term adds the missing sideways component.

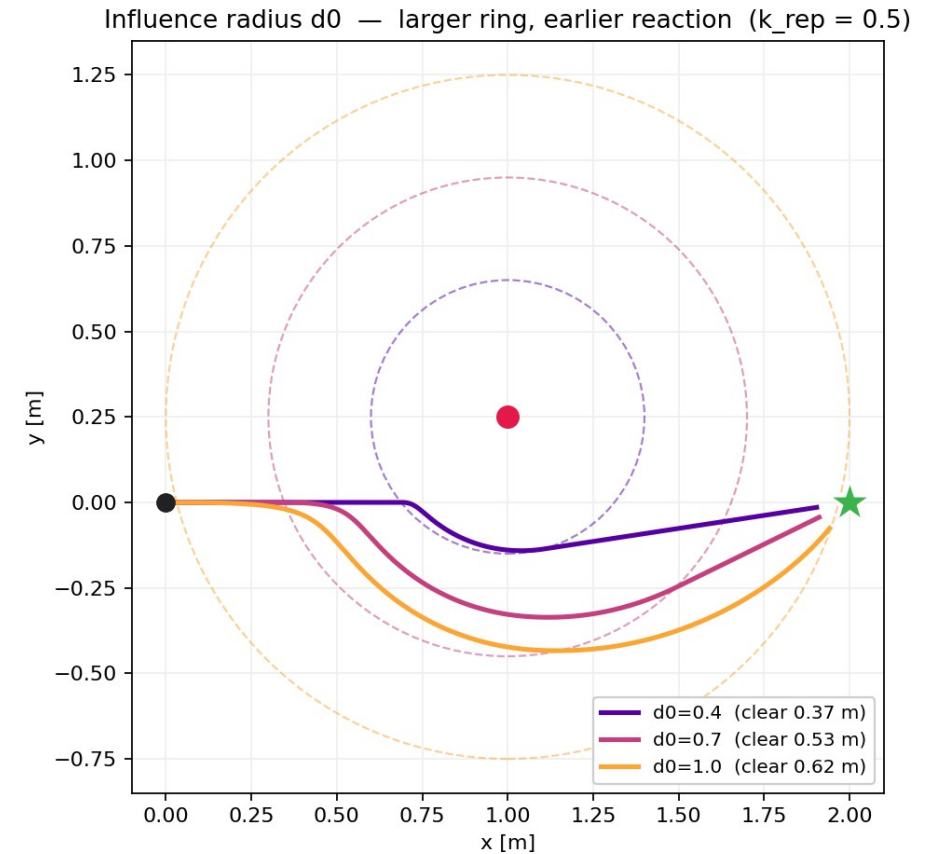
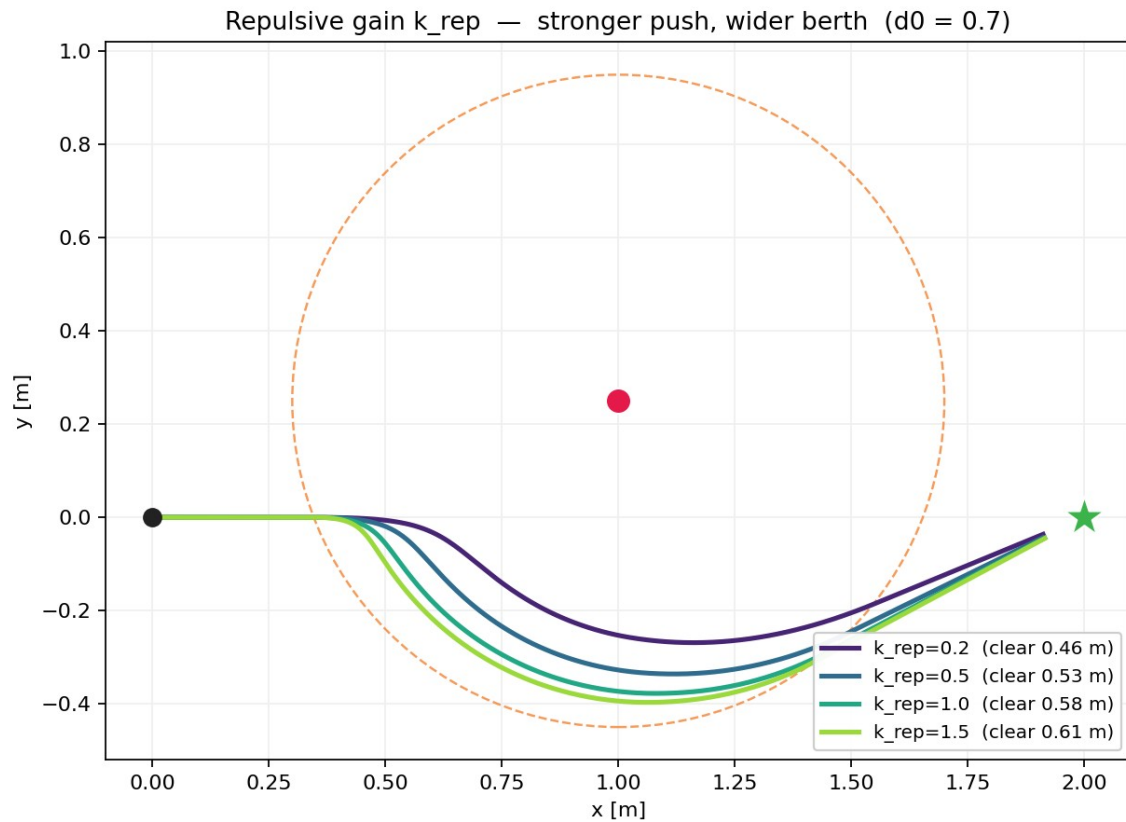
Limit: a sideways curl frees a saddle, but cannot climb out of the box canyon. A true minimum needs memory or a planner.



rotational field OFF — stalls



Configuring the field: k_{rep} and d_0

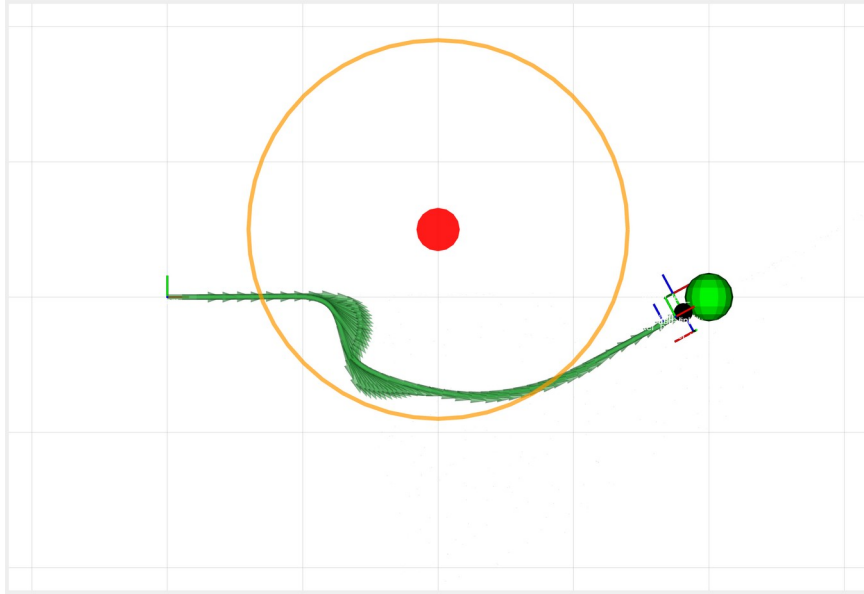


k_{rep} = how HARD it pushes. Clearance 0.46 → 0.61 m.

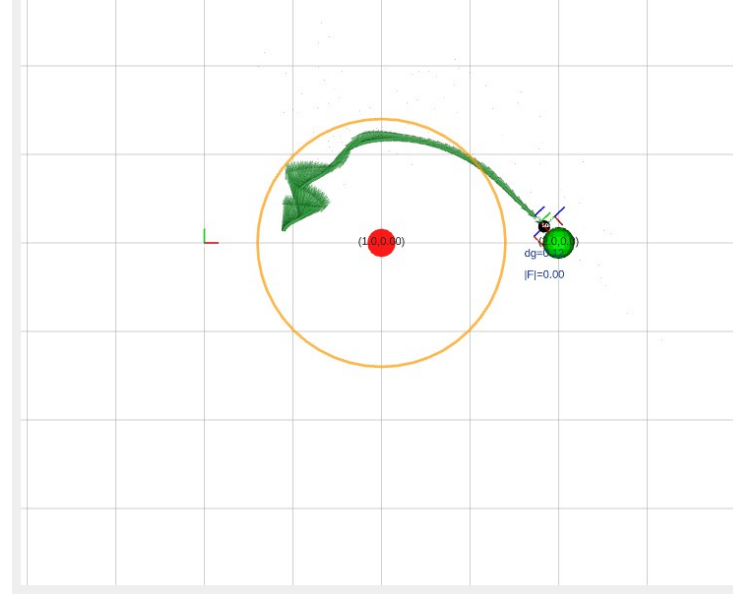
d_0 = WHEN it starts pushing. Clearance 0.37 → 0.62 m.

Both are one line in the YAML: edit, restart the node, compare in RViz.

What should your results look like



Real TB3 · scenario 1 · goes around



Real TB3 · scenario 4 · escaped

Measure and fill in:

XX m

Scenario 1

end pose from the log

XX m

Scenario 1

end position by tape

XX m

Scenario 2

stall point x

XX m

Scenario 3

where it stops

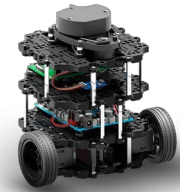
XX m

Scenario 4

stall or escape?

Robots drift: in our test, the log said 'goal reached' while the tape measured about 1.70 m of the 2.0 m. The field is computed in the robot's belief, not the truth.

Your Mission Today



20 min

Theory

Fields, forces, steering, equilibria



25 min

Simulation

Scenario builder + fake robot +
RViz



55 min

Robot experiments

Five scenarios + tuning on the
TB3

+ 20 min wrap-up and discussion



Bonus · Final Challenge: design a layout that traps the robot, then free it by changing ONE parameter — calculate, predict, verify!